Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum

Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı ve Anlatım Yöntemi
- 2. Kaynaklar ve Ön Koşullar
- 3. Yazılım Kalitesi
- 4. Neden İyi Tasarım? Yazılım Projelerinin Başarısı
- 5. Düşük Bağlaşım (Low Coupling)
- 6. Yüksek Uyum (High Cohesion)
- 7. Uyumun Ölçülmesi
- 8. İlgi Alanlarının Ayrılması
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Kapsamı ve Anlatım Yöntemi

Ders iki kısımdan oluşur (ekrandan, 00:10:00):

- Kısım 1: Gang of Four Tasarım Kalıpları. Dönemin yaklaşık üçte ikisi bu kısma ayrılır.
- Kısım 2: Code Smells and Refactoring (birim sınamaları ile). Dönemin kalan üçte biri bu kısma ayrılır.

Birinci kısım işin doğrusunu öğretir: nesneye dayalı bir tasarım nasıl iyi yapılır. İkinci kısım kötü işin nasıl yapıldığını, bunun nasıl fark edildiğini ve nasıl düzeltildiğini anlatır. Hocaya göre hem doğruyu öğrenmek hem de ne yapmamak gerektiğini öğrenmek önemlidir. İki kısım ayrı içeriklere sahiptir ama kesinlikle ayrık değildir, birbirini tamamlar.

Bilgisayar bilimlerinde, özellikle nesneye dayalı tasarım ve modelleme alanında “tasarım kalıpları” denince önce “dörtlü çete” (Gang of Four) lakaplı dört yazarın önerdiği kalıplar akla gelir. Bu kalıplar 1994’te Addison-Wesley’den çıkan *Design Patterns: Elements of Reusable Object-Oriented Software* kitabındadır. Zamanla literatüre yeni kalıplar da eklenmiştir, ama GoF kalıpları en temel kümedir. Analiz, tasarım ve kodlama alanlarında çalışanların bu kalıpları bilmesi büyük önem taşır. Ders notlarının tasarım kalıpları kısmı da bu kitaptan alınmıştır.

“Tasarım kalıbı” terimi aslında mimarlıktan gelir. Konunun önemini gösteren bir ölçü de üzerine yüzlerce kitap yazılmış olmasıdır. Bu kitaplardan yalnız GoF kitabı kalıbın tanımını ve içeriğini anlatır. Öteki kitaplar kalıbın çeşitli örnekler üzerinden kurulumunu ve gerçeklenmesini gösterir. Her iki türü de çalışmak yararlıdır.

1.1 Kalıpların derste örneklenmesi

Ders notları GoF’un anlatım tarzını, hatta anlatımlarını kullanır. Kalıp anlatıldıktan sonra en az bir örnek yapılır. Bazı kalıplar için aynı örneğin birden çok çözümü verilir:

1. Akla ilk gelen, sonradan kullanışsız olduğu ortaya çıkan çözüm.
2. Tasarım kalıbı kullanmadan, ama nesneye yönelimi doğru kullanmaya çalışarak elde edilen çözüm.
3. Tasarım kalıbı uygulanarak elde edilen çözüm.

Bu çözümler çeşitli açılardan karşılaştırılır. Bu ayrıntı her kalıp için yapılmaz, ama bir iki çok basit kalıp dışında her kalıbın en az bir tasarım kalıbı örneği gösterilir.

Refactoring kısmında Martin Fowler'ın kitabındaki kod kusurlarının ve refactoring eylemlerinin hepsi işlenmez. Hoca en yoğun kullanılanları ve birbirleriyle ilişki ağına bakarak en iç içe geçmiş bir alt kümeyi seçmiştir.

2. Kaynaklar ve Ön Koşullar

2.1 Kaynak kitaplar

Slayttaki kaynaklar (ekrandan, 00:10:00 ve 00:14:50):

- Kısım 1: *Design Patterns: Elements of Reusable OO Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994.
- Kısım 2: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999.
- Ek kaynaklar (elektronik ortamda bulunuyorlar):
  - *Design Patterns Explained: A New Perspective on Object-Oriented Design*, Alan Shalloway ve James R. Trott, Addison-Wesley, 2004 (2. baskı).
  - *Head First Design Patterns*, Elisabeth Freeman vd., O'Reilly, 2004.
  - *Design Patterns* (Wordware Applications Library), Christopher G. Lasater, Jones & Bartlett, 2006 (1. baskı).
  - *Design Patterns Java Workbook*, S. John Metsker, Addison-Wesley, 2002.
  - *Applied Java Patterns*, S. Stelting ve O. Maassen, Prentice Hall, 2002.

Hoca bunların zamanında bulabildiği ve örneklerini kullandığı kitaplar olduğunu, öğrencilerin çok daha fazlasını bulabileceğini söyledi. Refactoring için ise tek kitap vardır.

2.2 Nesneye yönelik programlama

Dersi başarıyla izlemek için herhangi bir güncel nesneye yönelik programlama dilinde bilgi gerekir (ekrandan, 00:14:50). Hoca örnek kodlarını Java ile gösterir. C# kullananlar bilir ki temel söz dizimi Java'ya çok benzer. Dillerin ileri düzey özellikleri değil, ortak genel özellikleri kullanılır; örneklerde istisna yönetimi (exception handling) bile yapılmaz. C++ gösterimi biraz farklıdır, ama yine de izlenebilir.

Kalıtım, soyut sınıflar ve arayüzler nesneye yönelimin temelleridir; bunlara hakim olmak gerekir. Hoca bunu bir ustalık-çıraklık benzetmesiyle açıkladı: bu ders ustalık kısmını kazandırmaya çalışır, ama çıraklık dönemi geçirilmeden ustalığa ulaşılamaz. Bu işin çıraklığı, nesneye yönelik dillerde çalışmış olmak, bazı problemleri çözmek için sınıf tasarımı yapıp bunu hayata geçirmektir.

2.3 UML sınıf şemaları

UML sınıf şemaları hakkında da bilgi gerekir (ekrandan, 00:14:50). Çok ayrıntılı bir birikim gerekmez, ama yazılım geliştirmenin herhangi bir aşamasında çalışanlar bu konuya mutlaka yatırım yapmalıdır. Önerilen kaynaklar:

- *UML Distilled*, 3. baskı (2003), Martin Fowler, Addison-Wesley. "Distilled" damıtılmış, özüt haline getirilmiş anlamındadır. Hoca UML şemalarına bu kitapla hakim olduğunu söyledi. Kitap, nesneye dayalı tasarımın nasıl daha düzgün yapılacağı konusundaki tecrübeleri de aktarır.
- *UML 2.0 in a Nutshell*, Dan Pilone ve Neil Pitman, O'Reilly. Nutshell serisi de özüt anlatmaya çalışır.
- Head First serisinin de bir UML kitabı vardır. Bu seri konuyu hiç bilmeyenlere en alt düzeyden başlayarak anlatır.

UML sınıf şemasının temel kuralları da nesneye yönelimin temellerine dayanır. Örneğin üye alanlar private, üye metotlar public tanımlanır. Şemada buna aykırı bir gösterim yapmak hatadır.

> **Hocanın vurgusu:** Nesneye yönelimin temellerini ve UML sınıf şemalarını bilmeyenler bu derse girişmemelidir.

2.4 Violet UML aracı

Hoca şema çizmek için Violet UML Editor adlı açık kaynak ve ücretsiz bir araç kullanır (01:10:00). Araç bir Java dosyasıdır; bilgisayarda Java çalışma ortamı kuruluysa çift tıklayarak çalışır. Koddan şema ya da şemadan kod üretmez, yalnız şema çizer, ama her türlü UML şemasını çizebilir. Şemalar aracın kendi biçiminde kaydedilir ve PNG resmine dönüştürülebilir. Sağdaki paletten öğe seçilip tıklanarak, sürüklenerek çalışılır. Hoca 0.21.1 sürümünü kullanıyor; yeni sürümler eski sürümün dosyalarıyla geriye doğru uyumlu olmadığı için eski sürümde kalmış.

Araçta bir sınıf çizilip üzerine çift tıklanınca özellikleri açılır: sınıf adı en üst bölmeye, alanlar ve metotlar alt bölmelere yazılır (ekrandan, 01:20:00).

3. Yazılım Kalitesi

3.1 İyi tasarım nedir?

Kalite başlı başına, çok bakış açısı olan bir konudur. Derste nasıl doğru bir tasarım yapılacağına ve bunun koda nasıl doğru dökületeceğine odaklanan iç kalite özellikleri ele alındı.

İyi bir tasarım yapmaya çalışmamızın nedeni iyi ve kaliteli bir yazılıma ulaşmaktır (ekrandan, 00:40:00). Hocanın sevdiği tanım şudur:

> Problem alanı hangi düzeyde parçalara ayrılabiliyorsa, en azından o düzeyde parçalara ayrılabilen bir tasarım “iyi” olarak nitelendirilebilir.

Bu tanım biraz özneldir, kişiden kişiye değişebilir; sayısal değerlerle karar verilebilecek bir tanım değildir. Bu yüzden daha nesnel ölçütlere ilerlemek gerekir.

3.2 Dış ve iç kalite özellikleri

Kaliteyi farklı biçimlerde gruplayanlar vardır. Hoca en basit gruplamayı tercih ediyor:

- Dış kalite özellikleri kullanıcıya yöneliktir: yazılımın doğruluğu, etkinliği (kaynakları mümkün olan en az düzeyde kullanarak verimli çalışması), kullanım kolaylığı.
- İç kalite özellikleri programcıya yöneliktir: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı (ekrandan, 00:40:00).

Nesneye yönelimi doğru kullanmak daha iyi bir tasarıma götürür; tasarım kalıpları kullanılarak daha da iyi sonuç elde edilir.

3.3 İç kalite özellikleri tek tek

Yeniden kullanılabilirlik (reusability). Yazılan kodun bir kısmını başka projelerde, başka problemlerin çözümüne katabilmektir. Aynı kodu sıfırdan yazmak yerine hazır kullanınca daha az çabayla daha çok iş yapılır. Ancak bu kazanç %60-80 düzeyinde değildir; hocanın okuduğu araştırmalara göre en fazla %30 civarındadır. Nedeni, kodu yeniden kullanılabilir hale getirmek için yapılan ek çabanın elde edilen kazancı azaltmasıdır.

İlgi alanlarının ayrılması (separation of concerns). Problem alanındaki alt gereksinimler, yapılacak işler, alt sorumluluklar tasarıma aynı düzeyde yansıtılmalıdır. Problem hangi düzeyde parçalara ayrılabiliyorsa tasarım da o parçalara birebir karşılık gelmelidir. Burada uyum (cohesion) özelliği devreye girer. Alt ilgi alanları ne kadar net ayrılıp tasarıma yansıtılırsa çözüm o kadar kaliteli olur.

Esneklik. Yazılan bir kod parçasının doğrudan yeniden kullanılması değil, benzer problemlere mümkün olan en az değişiklikle aktarılabilmesidir. Başka bir bakışla: gereksinimlerdeki değişimden kaçınılamaz. Değişiklik geldiğinde yazılımı bu gereksinimleri de karşılayacak biçimde değiştirmek için harcanan çabanın mümkün olduğunca az olmasıdır. Bu ikinci bakış bakım kolaylığını da tarif eder.

Hoca esnekliği şu örnekle açıkladı. Finans yazılımı ile sağlık yazılımının ortak noktası ne olabilir? İlk bakışta ayrı alanlardır. Ama finans yazılımı hisse senedi ya da döviz değerlerinin zaman içindeki değişimini anlık izler; buradan algoritmik ticaret (algotrading) işlemleri, örneğin bir eşiğin üstüne çıkınca al ya da sat gibi otomatik emirler tanımlanır. Sağlık yazılımı da hastaların hayati göstergelerini izler: en basit kan tahlilinde bile en az 10 satır değer vardır ve bunların zaman içindeki değişimi izlenir; daha acil bir durumda bir çocuğun ateşi yarım saatlik aralarla ölçülür. Değerin eşiğin üstüne çıkması da altına inmesi de kötüdür, bir bant aralığında seyretmesi gerekir. İki alanda da aynı iş yapılır; analiz ve grafikleme özellikleri yeniden kullanılabilir ya da kod küçük değişikliklerle bir alandan ötekine aktarılabilir.

Taşınabilirlik (portability). Aynı yazılımın farklı platformlarda kullanılabilmesi için yapılması gereken işlemlerdir. Örneğin Windows'ta çalışan bir uygulamayı macOS'a taşımak ya da Microsoft SQL Server kullanan bir yazılımı Oracle Database kullanacak biçimde taşımak.

Okunabilirlik ve anlaşılabilirlik. Bakım kolaylığı için önemlidir. İkisi farklıdır, çünkü bir şeyi okuyup anlamayabilirsiniz. Yazılımlar artık ekiplerce geliştirildiği için bir kişinin yazdığı kodu ekibin öteki üyelerinin, sınama ekibinin ve sonraki sürümler üzerinde çalışacak kişilerin en az çabayla anlayabilmesi çok önemlidir. Açık kaynak projelerde farklı altyapılardan, kültürlerden ve eğitimlerden gelen insanlar katkıda bulunur; birbirlerini anlayabilmeleri gerekir. Tasarım kalıpları ortak bir sözlük sağlayarak buna katkıda bulunur. Refactoring yaparken yalnız anlaşılabilirliği iyileştirmeye yönelik çabalar bile değerli refactoring çabalarıdır.

Sınanabilirlik (testability). Bir şeyi düzgün test edemiyorsanız büyük ihtimalle doğru tasarlayamamışsınızdır. Birim sınama konusunda dersin ilerleyen kısmında daha çok ayrıntı verilecektir.

Tasarım kalıpları kullanılarak bu iç kalite özelliklerini iyileştiren çözümler üretilecektir.

4. Neden İyi Tasarım? Yazılım Projelerinin Başarısı

4.1 Pratik gerekçe

İyi tasarım yapmaya mükemmeliyetçilikten değil, son derece pratik nedenlerle çalışılır: yazılım projeleri büyük oranda başarısızlığa uğrar (ekrandan, 00:41:30). Yazılım mühendisliğinin ve yazılım kalitesi konularının ortaya çıkışı da buna dayanır. 1950'ler ve 60'larda "yazılım krizi" (software crisis) diye ortaya konan durum gerçek ve süren bir olgudur. Belli problemler bilgi sistemleriyle çözüldükçe daha büyük problemlerin çözülmesi gerekir; ulaşılması gereken çıta sürekli yükselir.

Slayttaki gerekçeler (ekrandan, 00:41:30):

- Yazılım projeleri önemli oranda başarısızlığa uğramaktadır.
- Etken sayısı şüphesiz fazladır (insani ve teknik etkenler), ancak "tasarımın iyiliği" denetlenebilen bir etkendir.
- Oluşan bir hatayı düzeltmenin bedeli, yazılım hayat döngüsünde ilerledikçe üstel olarak artmaktadır.
- Yazılım geliştirme çabalarının çoğu bakım aşamasında harcanmaktadır.

Hata düzeltme bedelinin üstel artışı yazılımın temel dinamiğidir. Bu yüzden mümkün olduğunca iyi bir başlangıç yapıp iyi bir tasarımla başlamak, doğru kararlar vererek bunu kodlamaya taşımak gerekir. Tasarımın iyiliği çeşitli kalite ölçütleriyle ya da tasarım kalıpları kullanılarak denetlenebilir.

4.2 Başarı tanımı ve uluslararası veriler

Hoca eski bir sunumundan veriler gösterdi ve rakamların bugün de pek değişmediğini söyledi. Bir yazılım projesinin tam anlamıyla başarılı sayılması için bütçe, süre ve işlev açısından

bütün hedeflere ulaşmış olması gerekir: iş planının süresini ve bütçesini aşmadan, üzerinde anlaşılmış bütün işlevleri sağlayarak tamamlanmalıdır.

Dr. Dobb's Journal 2013 IT Project Success Survey verisi (ekrandan, 00:45:20): yüzden fazla şirketin en az bir projesi için doldurduğu bir ankettir. Grafik süreç türüne göre başarılı, zorlanılmış ve başarısız projelerin oranlarını gösterir. Süreç türleri:

- Geleneksel: şelale (waterfall) modeli.
- Ad-hoc: yazılım evinin kendi geliştirme sürecini kendisinin kurması. Literatürdekilerden farklı, kendi çalışma düzenine uygun özgün bir süreç ya da farklı süreçlerden kendine uygun kısımları seçerek oluşturduğu karma bir süreç olabilir. Hoca "öylesine" kelimesinin özensiz bir izlenim bıraktığını, ama durumun tam öyle olmadığını belirtti.
- Agile, Iterative, Lean.

Geleneksel ve ad-hoc süreçlerle çalışan yazılım evlerinde tam başarı oranı yaklaşık %50'dir. Çevik, yinelemeli ya da Lean süreçlerle çalışanlarda bu oran %65-70'e kadar çıkar.

Başarı tanımı esnetilirse, yani üç ölçütten birinde olumsuzluk olan ama ötekilerde hedefe ulaşılan projeler "zorlanılmış" proje sayılırsa, rakamlar iyileşir ve %90 gibi bir başarı oranına ulaşılır. Geri kalanlar zaten başarısızdır.

4.3 Türkiye'den bir çalışma

Çevik süreçler başarı oranını yükseltiyor göründüğü için Türkiye'de çevik çalışan yazılım evleriyle yapılmış bir çalışmaya bakıldı: E. Çetin ve P. O. Durdu, "Türkiye'de Çevik Yazılım Geliştirme Üzerine Bir İnceleme", UYMS 2014 (ekrandan, 00:48:20). Katılımcı sayısı azdır (toplam 35). Soru şudur: "Kurumunuzda gerçekleştirilen çevik projelerin başarı oranları nedir?" Pasta grafiğinin dağılımı:

Projelerin başarı oranı	Katılımcı sayısı	Yüzde
<%10	2	%6
%10-%25	4	%12
%26-%50	5	%14
%51-%75 (büyük olasılıkla)	13	%37
%76-%100	11	%31

Hoca 100'den 31'i çıkarıp 69 buldu ve "en fazla %69 başarı" diyerek bunun uluslararası %65-70 düzeyine yakın olduğunu, Türkiye'de de durumun aynı olduğunu söyledi. Buradan çıkan ders: işimizi mümkün olduğunca iyi yapmaya çalışıp kaliteye önem vermeliyiz.

> **Ders sonrası not:** Hoca grafikteki %69'u bir başarı oranı gibi okudu. Bu sayı büyük olasılıkla, çevik projelerinin başarı oranı %75 ve altında kalan katılımcıların payıdır: $6 + 12 + 14 + 37 = 69$. Başka bir deyişle katılımcıların %31'i projelerinin %76-%100'ünün başarılı olduğunu bildirmiştir. Yüzdeler sayılardan python ile doğrulandı ($2/35 \approx 5{,}7$, $4/35 \approx 11{,}4$, $5/35 \approx 14{,}3$, $13/35 \approx 37{,}1$, $11/35 \approx 31{,}4$).

5. Düşük Bağlaşım (Low Coupling)

5.1 Tanım

İyi tasarıma götüren en temel iki ilke düşük bağlaşım (low coupling) ve yüksek uyumdur (high cohesion). İlgi alanlarının ayrılması da önemli bir ilkedir, ama ilk ikisi cinsinden de tarif edilebildiği için ayrıca ele alınmaz.

Slayttaki tanım: bağlaşım, bir parçanın diğer parçalara bağımlılık oranıdır (ekrandan, 01:06:50). Hoca burada bir sorun gördü: tanımlanan kelimeyle aynı kökten kelimeler tanımda kullanılınca bir yumurta-tavuk problemi çıkar. Bu yüzden şöyle de söylenebilir: bağlaşım, bir parçanın diğer parçalarla ilişkide olma düzeyi ya da diğer parçalara ihtiyaç duyma düzeyidir.

Parça farklı düzeylerde olabilir: sınıf, alt sistem, paket, modül, bileşen. Derste sınıf düzeyinde bağlaşım ele alınır.

5.2 Bağımlılığın nedenleri

Bir sınıf diğerine şu durumlarda bağımlıdır (ekrandan, 01:09:20):

- Diğerinin hizmetlerinden yararlanıyorsa,
- İç yapısından haberdar olması gerekiyorsa,
- Çalışma prensiplerinden haberdar olması gerekiyorsa,
- Özelleşmiş veya genelleşmiş hali ise (kalıtım ilişkisi).

Bunların hemen hepsi UML sınıf şemasında bir okla gösterilir. İki sınıf arasında bir ok varsa, okun başladığı sınıf okun ucunun gittiği sınıfa bağımlıdır.

Diğer sınıfların sayısı arttıkça bağlaşım oranı artar. Burada önemli olan nesne sayısı değil sınıf sayısıdır.

Düşük bağlaşımın yararları (ekrandan, 01:09:20):

- Bir sınıfta yapılan değişikliğin geri kalan sınıfların daha azını etkilemesi.
- Yeniden kullanılabilirliğin artması.

Hoca slaytta yazmasa da düşük bağlaşımın esnekliği de artırdığını ekledi: bir sınıftaki değişikliğin ötekileri daha az etkilemesi, gereksinim değişikliklerine karşı daha verimli bir esneklik sağlar.

5.3 İlişkinin varlığı sayılır, niceliği değil

Hoca Violet UML'de A, B ve C sınıflarını çizdi: A'dan B'ye bire bir sahiplik, B'den C'ye bire bir sahiplik (ekrandan, 01:20:00). A'dan B'ye bire bir sahiplik, A sınıfında B türünden bir üye alan olduğu anlamına gelir. Bunu açıkça yazmaya gerek kalmadan bilmek gerekir.

İlişki bire çok olsaydı (çokluk *), A'da B türünden bir dizi ya da bir koleksiyon (örneğin bağlı liste) olurdu. Karede A sınıfının alanı `- b : B []` olarak yazıldı (ekrandan, 01:20:00). Koleksiyon sınıfları dile özgüdür, ama hemen her dilde karşılığı vardır.

Bağlaşım açısından bire bir ilişki ile bire çok ilişki aynı sayılır. A'da B türünden tek bir alan da olsa N tane de olsa, A'nın B'ye ihtiyacı vardır: A'nın kaynak kodu B'nin kodu olmadan başka bir yerde kullanılamaz. Aynı şey B ile C arasında da geçerlidir.

Kalıtım da bir bağlaşımdır. Kişi sınıfından kalıtımla türetilmiş bir Çalışan sınıfı Kişi'ye bağımlıdır; Java'da `public class Calisan extends Kisi` yazılır ve Kişi'nin kodu olmadan Çalışan'ın kodu kullanılamaz.

Kalıtım aslında daha "ağır sıklet" bir ilişkidir. Deneyimli programcılar ve tasarımcılar bir ilişkiyi kalıtım yerine sahiplik gibi bir ilişkiyle modelleyebiliyorsa bunu tercih eder, çünkü kalıtım en az esnek ilişki türüdür. A sınıfından bir nesne B sınıfından herhangi bir nesneyle ilişkilendirilebilir, çalışma anında bu ilişki koparılıp başka bir B nesnesiyle kurulabilir. Ama Kişi'den türetilen Çalışan her zaman bir kişidir; Kişi'nin bütün özelliklerini miras alır, üstüne kendi özelliklerini ekler ve bu değiştirilemez. Kalıtımın yanlış kullanımına uygun olmayan çözüm örneklerinde değinilecektir.

Yine de bağlaşım açısından hepsi birer ilişki sayılır: B'den C'ye, A'dan B'ye ya da Çalışan'dan Kişi'ye. İlişkinin niceliğine değil varlığına bakılır: varsa 1, yoksa 0.

5.4 İki tasarımın karşılaştırılması: zincir ve yıldız

Aynı problemi çözen iki tasarım olduğu varsayıldı (ekrandan, 01:30:00):

```
Tasarım I (zincir):  A --> B --> C
Tasarım II (yıldız): A --> B
                     A --> C
```

Karşılaştırmayı benzer koşullarda yapmak için bütün ilişkiler bire bir alındı. Soru: hangi tasarım daha düşük bağlaşıma sahiptir?

Önce sınıf bağlaşımları yazılır:

- Tasarım I: bağ(A) = 1 (B'ye ihtiyaç), bağ(B) = 1 (C'ye ihtiyaç), bağ(C) = 0.

- Tasarım II: bağ(A) = 2 (B ve C'ye ihtiyaç), bağ(B) = 0, bağ(C) = 0.

Tasarımı bir bütün olarak değerlendirmek için ilk akla gelen ölçüt toplam bağlaşımdır:

- tBağ(I) = 1 + 1 + 0 = 2.
- tBağ(II) = 2 + 0 + 0 = 2.

Toplam bağlaşım iki tasarım arasında fark göstermedi.

Hoca ikinci ölçütü bir benzetmeyle getirdi: savaş planlaması gibi yüksek riskli durumlarda birden çok hareket tarzı varsa en az kayba yol açacak olan tercih edilir. Buna göre bağlaşımın yüksek olduğu yerlerden kaçınmak için maksimum bağlaşım tanımlanır: sistemdeki en çok bağlaşıma sahip sınıfın bağlaşımı sistemin bağlaşımı sayılır.

- maxBağ(I) = 1 (A ve B'nin ikisi de 1).
- maxBağ(II) = 2.

Maksimum bağlaşıma göre Tasarım I daha iyidir. Bu makul bir karar olabilir.

5.5 Dolaylı ilişkiler

Ancak bir şey gözden kaçtı: dolaylı ilişkiler. Tasarım I'de A'nın kodunu yeniden kullanmak için yalnız B'nin kodu yetmez; B de C'ye ihtiyaç duyduğu için A, C'ye dolaylı olarak bağımlıdır. Dolaylı ilişkiler de hesaba katılırsa daha sağlam bir hesap yapılmış olur.

Dolaylı ilişki doğrudan ilişkiyle aynı keseye konmamalıdır; A'nın bağlaşımına C için 1 değil 0,5 eklenir. Hoca şemada dolaylı ilişkiyi kesikli çizgiyle gösterdi. Kesikli çizgi UML'de aslında bu anlama gelmez (arayüz gerçekleme gibi anlamlarda kullanılır), hoca bu amaç için kullanıyor.

> **Ders sonrası not:** Hoca kesikli çizginin UML'de arayüz gerçekleme anlamına geldiğini söyledi. Büyük olasılıkla daha doğrusu şudur: arayüz gerçekleme kesikli çizgi ve içi boş üçgen uçla gösterilir; açık uçlu kesikli ok ise UML'de bağımlılık (dependency) ilişkisidir. Karede hocanın Violet'ta "Depends on" aracını seçtiği görülüyor (ekrandan, 01:32:15).

Yeni değerler (ekrandan, 01:32:15 ve 01:46:10):

- Tasarım I: bağ(A) = 1 + 0,5 = 1,5; bağ(B) = 1; bağ(C) = 0. tBağ(I) = 2,5; maxBağ(I) = 1,5.
- Tasarım II: dolaylı ilişki yoktur. tBağ(II) = 2; maxBağ(II) = 2.

Şimdi sonuçlar çelişir:

Ölçüt	Tasarım I	Tasarım II	Daha iyi
Toplam bağlaşım	2,5	2	II
Maksimum bağlaşım	1,5	2	I

5.6 Çelişen ölçütlerde karar

Hangisi seçilmeli? Keyfi bir seçim ya da yazı-tura yerine şöyle bir yol önerildi. Bir yazılım evi şimdiye kadar gerçekleştirdiği projelerin bağlaşım ölçümlerini yapar. Bu ölçümler elle yapılmak zorunda değildir: literatürde tanımlı çeşitli coupling metrikleri vardır ve bunları kaynak kod üzerinden ya da Java için class dosyaları üzerinden ölçen araçlar bulunur.

Sonra geçmiş projeler bir başarı ölçütüyle karşılaştırılır. Bu ölçüt önceki saatte konuşulan başarı tanımı olabilir ya da örneğin bin kod satırı başına bulunan hata sayısı gibi bir metrik olabilir. Yaklaşık aynı ölçekte ve yakın sürelerde tamamlanmış projeleri karşılaştırmak yararlıdır. Hangi projeler daha kaliteliyse onların toplam ve maksimum bağlaşımlarına bakılır. Böylece "bizim problemlerimizde toplam bağlaşım kaliteye daha çok etki ediyor" ya da "maksimum bağlaşım daha çok etki ediyor" denip karar verilir.

Hoca bir de şu itirazın haklı olduğunu söyledi: "A ne, B ne, C ne?" Tartışma iki tasarımın da aynı problemi çözdüğü varsayımıyla yürütüldü. Bu sınıflar problem alanını oluşturan varlıklarsa, gerçek problemde ilişki nasıl kurulmuşsa tasarımda da öyle kurulmalıdır; bağlaşım hesabına

bakılmaz. Bu tartışma ancak sınıflar problem alanına değil de bir altyapı katmanına aitse ve iki tasarım da işi görüyorsa anlamlıdır.

6. Yüksek Uyum (High Cohesion)

6.1 Tanım

Slayttaki tanım: uyum, bir parçanın sorumluluklarının birbirleri ile uyumlu olma oranıdır (ekrandan, 01:40:00). Bu da bir yumurta-tavuk tanımı olduğu için hoca açtı: bir parçanın sorumluluklarının aynı ya da çok yakın ilgi alanlarında olması durumudur.

Bir varlığın tek bir rolü ya da birden çok rolü olabilir; aynı rolün getirdiği birden çok sorumluluk da olabilir. Sorumluluklar atanırken birbiriyle ilişkili, birbirine yakın, aynı alanlarda olmaları istenir. Aksi halde o sınıf için bir tür "şizofrenik" durum oluşur.

6.2 Yazılım geliştirici benzetmesi

Bir şirkette yazılım geliştirici olarak çalışan birine pozisyonuyla ilgili işler verilir. "Sekreterimiz hastalandı, işini bırak, şirketin telefonlarına bak" denmez. Bilgisayarı arızalandığında, sorunu anlayıp hızlıca bakabilecek olsa bile, onarım sorumluluğu onda olmamalıdır; örneğin tek RAM çubuğu bozulduysa parça değişimiyle IT ekibinden biri ilgilenir. Bu farklı sorumluluklar yüklenirse o kişiden alınan verim düşer: telefonlara bakarken yazdığı kodda acil bir sorun çıkarsa ne yapacak? Farklı alanlarda sorumluluklar atanan bir varlık verimsiz çalışır; tasarımınız böyleyse son derece uygun olmayan bir tasarım ortaya koymuş olursunuz.

6.3 Yüksek uyumun yararları

Slayttaki yararlar (ekrandan, 01:40:00):

- Sınıfın anlaşılma kolaylığı artar: tek bir problem alanında sorumlulukları olduğu için yalnız o alanın bilgisiyle anlaşılabilir.
- Yeniden kullanılabilirlik artar: odaklanmış, sınırları iyi tanımlanmış bir sorunu çözdüğü için o sorunun geçtiği her yerde kullanılabilir.
- Bakım kolaylığı artar: daha rahat anlaşılan şeyin bakımı da daha kolaydır.
- Sınıfın değişikliklerden etkilenme olasılığı düşer: gereksinimdeki bir değişiklik sınıfın odaklandığı alanın dışındaysa o sınıfın kodu boşuna değişmez.

Bağlaşım ve uyum genelde aynı yönde rehberlik eder. Düşük bağlaşım getiren bir tasarım yüksek uyumu, yüksek bağlaşım getiren bir tasarım ise düşük uyumu beraberinde getirir (ekrandan, 01:50:00).

6.4 Veritabanı örneği: bağlaşım ile uyumun ters düştüğü durum

A→B→C tasarımı seçilmiş olsun ve bu sınıflar problem alanındaki varlıklar olsun. Bu varlıkların nesneleri bir veritabanına yazılacak ve oradan okunacaktır. (Klasik şemalarda veritabanı silindirle gösterilir; Violet'ta böyle bir sembol olmadığı için hoca başka bir şekil kullandı.) İki tasarım karşılaştırıldı (ekrandan, 01:46:10 ve 01:58:45):

```
Tasarım I:
  A --> B --> C            (iş mantığı katmanı)
  VT <>-- A, B, C          (VT, A, B ve C nesnelerine sahip; çoklu olabilir)
  VT ---> veritabanı       (sistem katmanı)

Tasarım II:
  A --> B --> C
  A <--> veritabanı
  B <--> veritabanı
  C <--> veritabanı
```

Tasarım I'de veritabanı işlemlerinden sorumlu ayrı bir VT sınıfı vardır. A, B, C nesnelerini veritabanına yazma ve okuma işleri bu sınıfa yaptırılır. Nesneler birden çok olabileceği için

ilişki elmaslı (sahiplik) çizildi. Fiziksel veritabanı A, B, C'yi doğrudan bilmeyebilir; VT sınıfı bir soyutlama yapabilir ya da yapmayabilir, bu tartışmanın dışındadır.

Tasarım II'de her nesne kendisini veritabanına yazmaktan ve okumaktan sorumludur. Buradaki ilişkiler çift yönlüdür: C kendini veritabanına yazmak için erişim bilgilerine sahip olmalı, veritabanında da C'nin kayıtları tanımlanmış olmalıdır. UML şemalarında okların yönü (tek ya da çift yönlü olması) önemlidir.

Hoca öğrencilere hangi tasarımı seçeceklerini sordu ve Zoom tepkileriyle oylama yaptı: Tasarım I'e 3, Tasarım II'ye 4 oy geldi (ekrandan, 01:58:45).

Önce bağlaşım açısından değerlendirme: Tasarım I'de VT sınıfı işin içine girince VT'den A, B ve C'ye üç yeni bağımlılık oluştu. Önceki toplam bağlaşım (2 ya da 2,5) üç artar. Tasarım II'de ise A, B, C'nin veritabanı hakkında bilgili olması yeter; veritabanının kendisi bir sınıf olmadığından bağlaşıma sayılmaz. Bağlaşım açısından Tasarım II çok daha iyi görünür.

Buna rağmen hoca Tasarım II'yi tercih etmiyor. Normalde bağlaşım ve uyum aynı yönde tetiklenir, ama bu örnek ikisinin ters düştüğü bir örnektir. Herhangi bir bilgi sisteminde kayıtların kalıcılığı gerekir: ya bir veritabanına ya bir dosya sistemine bir şeyler kaydedilir. Tasarım II'de iş mantığı sınıflarına veritabanı alt sistemiyle ilgili sorumluluklar da verilir. Bu "gariban" sınıflar iş mantığıyla ilgili görevlerini mi yapsın, veritabanıyla ilgili görevlerini mi? Birbiriyle uyumsuz görev alt kümelerine sahip olurlar.

Somut sonuç değişiklik anında görülür. Örneğin Microsoft SQL Server'ın lisans ücreti pahalı geldi ve MySQL'e geçilecek. Ya da varlıklar klasik kayıt mantığından çok bir harita bilgi sistemindeki yol noktaları ya da Facebook, LinkedIn gibi sosyal ağlardaki kişiler arası ilişkiler gibi verilerse ilişkisel olmayan bir veritabanına geçmek gerekebilir. Tasarım II'de veritabanı değişince veritabanıyla ilişkili bütün sınıflarda kod değişikliği gerekir. Tasarım I'de ise yalnız VT sınıfının kodu değişir; iş mantığını oluşturan A, B, C sınıflarına dokunmak gerekmez.

> **Hocanın vurgusu:** Bu çok önemli bir avantajdır. Doğru çözüm Tasarım I'dir: yüksek uyuma sahip çözüm tercih edilir (ekrandan, 01:58:45).

Tasarım I aslında bir katmanlı mimari kurar. Sistem iki katmandan oluşur: iş mantığı katmanı (A, B, C) ve sistem katmanı, daha özel söylenirse veritabanı katmanı. Tasarım II'de katman ayrımı yoktur: sınıfların hepsi hem problem alanıyla hem veritabanıyla uğraşır. Tasarım I ile ilgi alanları da birbirinden daha iyi ayrılmış olur.

7. Uyumun Ölçülmesi

Bağlaşım kolay sayılır: UML sınıf şemasında oklara, kaynak kodda üye alan sahipliğine, metot parametrelerine bakılır. Uyumun nasıl belirleneceği ise kolay bir problem değildir, ama çözülmemiş bir problem de değildir. Literatürde çeşitli cohesion ölçütleri vardır. Statik kod analizine dayananlar genelde şunu yapar (ekrandan, 02:05:00).

Hoca bir sınıfı Paint.NET'te şöyle çizdi: kareler sınıfın üye alanları, eşkenar dörtgenler metotlarıdır. Sınıfın 3 üye alanı ve 4 metodu vardır. Her metottan kullandığı alana bir çizgi çizilir:

```
Alanlar:  F1      F2      F3
Metotlar: M1  M2  M3      M4

Kullanımlar: M1-F1, M1-F2, M2-F1, M2-F2, M3-F2, M4-F3  (toplam 6)
```

Graf tam bağlı olsaydı, yani her metot her alanı kullansaydı, $4 \times 3 = 12$ kullanım olurdu. Çizilen 6'dır:

$$\text{uyum} = \frac{6}{12} = 0{,}5$$

Uyum %50'dir; çok iyi değildir. Bu sınıfa bir refactoring eylemi uygulanabilir: sınıf iki ayrı sınıfa bölünür (ekrandan, 02:07:55).

- Birinci sınıf (kırmızı): 2 alan, 3 metot. En çok $2 \times 3 = 6$ çizgi olabilir; 5 çizilmiştir. uyum $= 5/6 \approx 0{,}83$, yani %83.

- İkinci sınıf (sarı): 1 alan, 1 metot. En çok 1 çizgi olabilir; 1 çizilmiştir. uyum $= 1/1 = 1{,}0$, yani %100.

Sistemin ortalama uyumu:

$$\text{ort. uyum} = \frac{0{,}83 + 1{,}0}{2} = \frac{1{,}83}{2} \approx 0{,}91$$

Tek sınıfken uyum 0,5 idi; refactoring sonrası ortalama uyum %90'ın biraz üstüne çıktı. Tasarım uyum açısından çok iyi bir yere taşınmış oldu.

> **Ders sonrası not:** Hoca 0,83'ü yuvarlanmış haliyle kullanıp 0,91 buldu. Kesir olarak hesaplanırsa $(5/6 + 1)/2 = 11/12 \approx 0{,}917$, yani yaklaşık %92 çıkar (python ile doğrulandı). Sonuç değişmez: uyum %90'ın üstüne çıkmıştır.

Burada yalnız doğrudan kullanımlar sayıldı. Dolaylı kullanımlar da sayılabilir; doğrudan ve dolaylı ilişkilerin sayılıp sayılmaması, aynı katsayıyı alıp almaması gibi çeşitlemeler vardır. Hoca bunları bağlaşımda epey tartışıldığı için ayrıca tartışmadı. Uyum da çok önemli bir ölçüttür ve literatürde onu sayısal göstergelerle ifade eden ölçütler vardır.

> **Ek bilgi (derste anlatılmadı):** Derste gösterilen metot-alan erişim oranı, literatürdeki uyum metriklerinin (örneğin LCOM, "Lack of Cohesion in Methods" ailesi) temel fikrine büyük olasılıkla yakındır. Bu metrikler de bir sınıfın metotlarının ortak alanları ne ölçüde paylaştığına bakar.

8. İlgi Alanlarının Ayrılması

Dersin son slaytı ilgi alanlarının ayrılmasıdır (ekrandan, 02:00:00). Slayttaki maddeler:

- İlgi alanlarının ayrılması ile daha iyi bir tasarıma ulaşılması: yazılımın sadece belirli bir amaç, kavram veya hedef ile ilgili kısımlarının tanımlanabilmesi, birleştirilebilmesi ve değiştirilebilmesi yeteneğine işaret eder.
- Sorunu parçalayarak fethet ve parçaları yeniden kullan.
- Her programlama yaklaşımı, sorunu parçalara bölmek için yollar sunar.
- Parçaların düşük bağlaşıma ve yüksek uyuma sahip olması istenir.

Hoca veritabanı örneğinde yaptığı şeyin de bir ilgi alanlarının ayrılması olduğunu belirtti: iş mantığı ile kalıcılık işleri ayrı sınıflara ve ayrı katmanlara ayrılmıştır.

Örnek Sorular

Soru 1 (derste sözlü, 00:36:58): Bir finans yazılımı ile bir sağlık yazılımının ortak noktası ne olabilir? Bu, esneklik ve yeniden kullanılabilirlik açısından ne anlama gelir?

> **Çözüm (derste verildi):** İki yazılım da zaman içinde değişen değerleri izler: finansta hisse ya da döviz değerleri, sağlıkta kan değerleri ya da ateş. İki alanda da değerin eşiklere göre (finansta al-sat emri, sağlıkta bir bant aralığı) izlenmesi gerekir. Bu yüzden izleme, analiz ve grafikleme kısmı yeniden kullanılabilir ya da küçük değişikliklerle bir alandan ötekine aktarılabilir.

Soru 2 (ekrandan, 01:30:00 ve 01:32:15): Aynı problemi çözen iki tasarım veriliyor. Tasarım I: A→B, B→C. Tasarım II: A→B, A→C. Bütün ilişkiler bire birdir. Her tasarım için sınıf bağlaşımlarını, toplam bağlaşımı ve maksimum bağlaşımı, önce yalnız doğrudan ilişkilerle, sonra dolaylı ilişkileri 0,5 ağırlıkla katarak hesaplayın. Hangi tasarım seçilmeli?

> **Çözüm (derste verildi):**
>
> - Doğrudan ilişkiler: Tasarım I'de bağ(A) = 1, bağ(B) = 1, bağ(C) = 0, tBağ = 2, maxBağ = 1. Tasarım II'de bağ(A) = 2, bağ(B) = bağ(C) = 0, tBağ = 2, maxBağ = 2.
> - Dolaylı ilişkiyle: Tasarım I'de A, C'ye dolaylı bağımlıdır; bağ(A) = 1,5, tBağ = 2,5, maxBağ = 1,5. Tasarım II değişmez.

- Toplam bağlaşıma göre II, maksimum bağlaşıma göre I daha iyidir. Karar, firmanın geçmiş projelerinde hangi ölçütün kaliteyle daha çok ilişkili olduğuna bakılarak verilir. Sınıflar problem alanındaki varlıklarsa ilişki gerçek dünyadaki ilişkiye göre kurulur.

Soru 3 (ekrandan, 01:46:10 ve 01:58:45): A→B→C sınıflarının nesneleri veritabanına kaydedilecek. Tasarım I'de ayrı bir VT sınıfı A, B, C nesnelerini veritabanına yazar ve okur. Tasarım II'de her sınıf kendini veritabanına yazar ve okur. Hangi tasarımı seçersiniz, neden?

> **Çözüm (derste verildi):** Tasarım II'nin bağlaşımı daha düşüktür, çünkü Tasarım I'de VT'den A, B, C'ye üç bağımlılık eklenir. Ama Tasarım II'de iş mantığı sınıfları veritabanı sorumluluğu da taşır ve uyumları düşer. Veritabanı değişirse (örneğin SQL Server'dan MySQL'e ya da ilişkisel olmayan bir veritabanına geçiş) Tasarım II'de bütün sınıflar, Tasarım I'de yalnız VT sınıfı değişir. Doğru çözüm yüksek uyumlu Tasarım I'dir; iş mantığı katmanı ve veritabanı katmanından oluşan katmanlı bir mimari kurar. (Derste oylamada Tasarım I'e 3, Tasarım II'ye 4 oy çıktı.)

Soru 4 (ekrandan, 02:05:00 ve 02:07:55): Bir sınıfın 3 üye alanı ve 4 metodu var; metotlar alanları toplam 6 kez kullanıyor. Sınıfın uyumu nedir? Sınıf, 2 alan-3 metot-5 kullanım ve 1 alan-1 metot-1 kullanım olan iki sınıfa bölünürse ortalama uyum ne olur?

> **Çözüm (derste verildi):** Tek sınıfın uyumu $6/(4 \times 3) = 6/12 = 0{,}5$. Bölünmeden sonra birinci sınıfın uyumu $5/(3 \times 2) = 5/6 \approx 0{,}83$, ikincinin $1/1 = 1{,}0$. Ortalama uyum $(0{,}83 + 1{,}0)/2 \approx 0{,}91$ (kesirle tam değer $11/12 \approx 0{,}92$). Bölme refactoring'i uyumu %50'den %90'ın üstüne çıkarmıştır.

Dersin Toparlaması

Bu derste dersin iki kısmı (GoF tasarım kalıpları ile kod kusurları ve refactoring), kaynakları ve ön koşulları tanıtıldı. Ardından iyi tasarımın neden gerektiği tartışıldı: yazılım projeleri yüksek oranda başarısız olur, hata düzeltme bedeli hayat döngüsünde üstel artar ve tasarımın iyiliği denetlenebilen bir etkendir. Kalite, kullanıcıya yönelik dış özellikler ve programcıya yönelik iç özellikler olarak ikiye ayrıldı; ders iç kalite özellikleriyle (yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, anlaşılabilirlik, sınanabilirlik, bakım kolaylığı) ilgilenir.

Dersin ağırlık noktası iyi tasarımın iki temel ilkesiydi. Düşük bağlaşım sınıflar arası bağımlılığı, ilişkinin varlığına bakarak ölçer; toplam bağlaşım, maksimum bağlaşım ve dolaylı ilişkiler farklı sonuçlar verebilir. Yüksek uyum bir sınıfın sorumluluklarının aynı ilgi alanında toplanmasıdır ve metot-alan kullanım oranıyla ölçülebilir. İki ilke genelde aynı yönü gösterir; ters düştüklerinde, veritabanı örneğinde olduğu gibi, yüksek uyumlu ve katmanlı çözüm tercih edilir. Bir sonraki derste GoF kitabı üzerinden tasarım kalıplarına giriş yapılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.